Haskell Tarea

Elaborar un mapa conceptual de los Paradigmas de Programacion
Representativa.

Modelo de Programacion Funcional

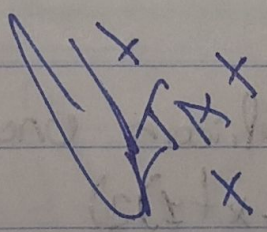
el modelo de Programacion Funcional es un estilo de Programa
cion declarativo que trata el calculo como la
evaluacion de funciones matematicas y evita cambiar datos
o estados.

en lugar de darle al ordenador una lista paso a paso de
ordenes (como en la Programacion tradicional), este modelo se
concentra en que se quiere calcular usando funciones Puras.

Conceptos Clave

- Funciones Puras: Son bloques de código que, si reciben
los mismos datos de entrada, siempre devuelven exactamente
el mismo resultado. No modifican nada fuera de ellas ni
causan efectos secundarios

- Funciones de orden superior: Son funciones que pueden recibir
otras funciones como datos de entrada o devolver
funciones como resultado



Calculo lambda de Alonzo Church

El calculo lambda es un sistema formal creado por el matematico Alonzo Church en la decada de 1930 para estudiar de manera abstracta las funciones, la recursión y la aplicacion de operadores.

A menudo se le describe como el lenguaje de Programacion mas pequeño del mundo por que no necesita numeros. Variables booleanas ni tipos de datos complejos; todo se construye a base de funciones.

Conceptos claves

- Variables: Representan los datos de entrada o nombres semanticos (por ejemplo, X, Y, Z)
- Abstraccion (λ): es la creacion de una funcion anonima. Se escribe con la letra griega λ seguida por un parametro, un punto y un cuerpo (por ejemplo λ x.x+1).
- Aplicacion: Es el acto de aplicar una funcion a un argumento. (por ejemplo. (λx.x+1)2)

Caracteristicas	Notacion Infija	Notacion Prefija	Notacion Postfija
Posicion del operador	Antes de los argumentos (f x y)	entre dedos argumentos (x f y)	Despues de los argumentos (x f)
Comportamiento Por defecto	es el estilo estandar de Haskell Para la mayoria de las funciones	exclusivo de operadores simbolicos (como +, *, ++).	No existe de forma nativa en la sintaxis de Haskell.
Conversion de estilo	N/A	Se vuelven Prefijas envolviendose entre Parentesis: (+) 2 3.	N/A
uso de funciones alfanumericas	se describen normal div 10 2.	se Vuelven infijas usando comillas invertidas (backticks) 10 `div` 2.	N/A
¿como simularlas?	N/A	N/A	se Pueden simular usando el operador de aplicacion Inversa (&) de Data. Function.
Ejemplo Basico	mod 10 3 (Resultado 1)	10 +3 Resultado 13	10 & succ Resultado 11

2.3 Intervalos en Haskell

¿Que son los Intervalos?

Los Intervalos en Haskell son una forma de generar listas de valores que siguen una secuencia determinada. Permiten crear conjuntos de numeros de manera sencilla sin tener que escribir cada elemento manualmente. Se utilizan principalmente con numeros enteros y decimales.

- Permiten generar secuencias de numeros
- Pueden avanzar de uno en uno o mediante saltos
- Se Pueden establecer limites Iniciales y finales
- Facilitan la creacion de listas
- Se Pueden combinar con funciones como Sum, length y ma

3 Poradores en Haskell

los oPeradores en Haskell Son Simbolos o funciones que Permiten realizar o Peraciones entre Valores y exPresiones. Se utilizan Para realizar calculos matematicos, comparar datos y combinar condiciones logicas.

Haskell Permite utilizar operadores de manera infija Centre dos valores y Prefija antes de los valores.

Evaluacion Perezosa

la evaluacion Perezosa (Lazy Evaluation) es una caracteristica de Haskell que consiste en no calcular una expresion hasta que se requerido sea realmente necesario.

Esto significa que Haskell puede trabajar con datos que todavia no han sido calculados completamente y evaluar unicamente las partes que necesita.

- No calcula valores innecesariamente
- Permite trabajar con listas infinitas
- Puede ahorrar recursos computacionales
- Evalua las expresiones cuando se necesitan
- Facilita la creacion de estructuras de datos grandes o infinitas